PW04 - Classification Trees and Random Forests

Machine Learning (FTP_MachLe) - MSE HES-SO
Noé Berdoz and Michael Strefeler
12 October 2026

Work time: We spent about 8h on this PW04. Exercise 1 took about 3h, exercise 2 about 2h30 and exercise 3 about 1h. Creating the report took about 1h30.

Bonus request

We would like to apply for the bonus because we did a bit more than what was asked.

- We added two charts of our own. The first one shows the median spectrum and the 25-75 % band of each class (Q1.2). The second one shows the training and test accuracy of a tree for every depth from 1 to 50 (Q1.4).
- We checked the teacher's saved figures against each other and found that the scatter plot and the heatmaps cannot come from the same randomized search (Q2.7).
- We compared the random forest with the decision tree that also uses class_weight='balanced', which is the fair comparison (Q2.11a).
- We trained Gradient Boosting with subsample 0.5 and 1.0 and compared their accuracy and training time (Q3.1).

1 Classification trees

Q1.1 *At which frequencies does the electrical activity mostly occur?*
Most of the electrical activity occurs at low frequencies. In all three states, the amplitude is highest below 10 Hz, drops quickly between 10 and 20 Hz, and is close to zero above 30 Hz.

The amplitude between 1 and 20 Hz accounts for 84% of the total for 'w' and 91% for 'n'. The peak depends on the state: 'w' is highest at 1 Hz, 'n' has a broad bump from 2 to 9 Hz, and 'r' has a sharp peak at 8 Hz.

Q1.2 *Can you easily distinguish between classes visually? What can you say about the inter- vs intra-class variability?*

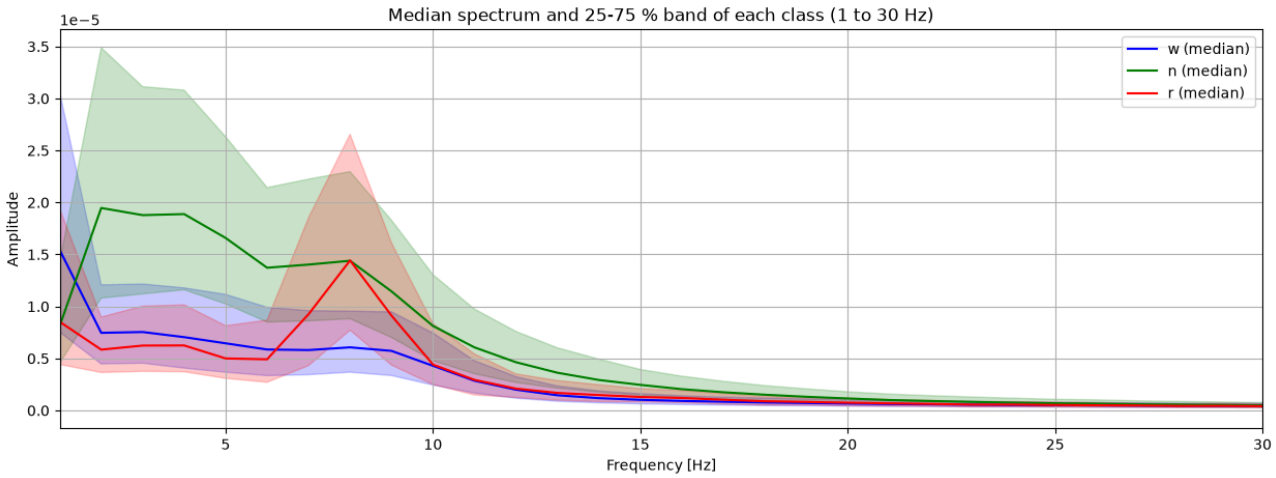


Figure 1 - Median spectrum and 25–75 % band of each class (1 to 30 Hz).

It is difficult to distinguish them because the intra-class variability is high. The figure above shows the median spectrum of each class (line) and the band that holds the middle 50 % of the rows (25-75 %). Even though the medians are different, the bands overlap at almost all frequencies. The band of 'w' overlaps those of 'n' and 'r' everywhere, and 'n' and 'r' only separate between 2 and 5 Hz. This means the variations within a single class are almost as large as the differences between the classes, making it hard to classify a single sample.

Q1.3 *Describe both of these criteria. What does a gini impurity of 0 means? What does an entropy of 1 mean?*
Both criteria measure the "impurity" (mix) of classes within a node.

- Gini impurity represents the likelihood of misclassifying an element if it were randomly assigned a class based on the node's distribution. A value of 0 indicates a perfectly pure node, where all observations belong to a single class.
- Entropy measures the information uncertainty in bits. For a two-class problem, an entropy of 1 signifies maximum uncertainty, occurring when both classes are equally distributed (50/50).

Q1.4 *This problem suffers from a common issue in machine learning. What is this problem called? What could be its causes? How can it be resolved?*

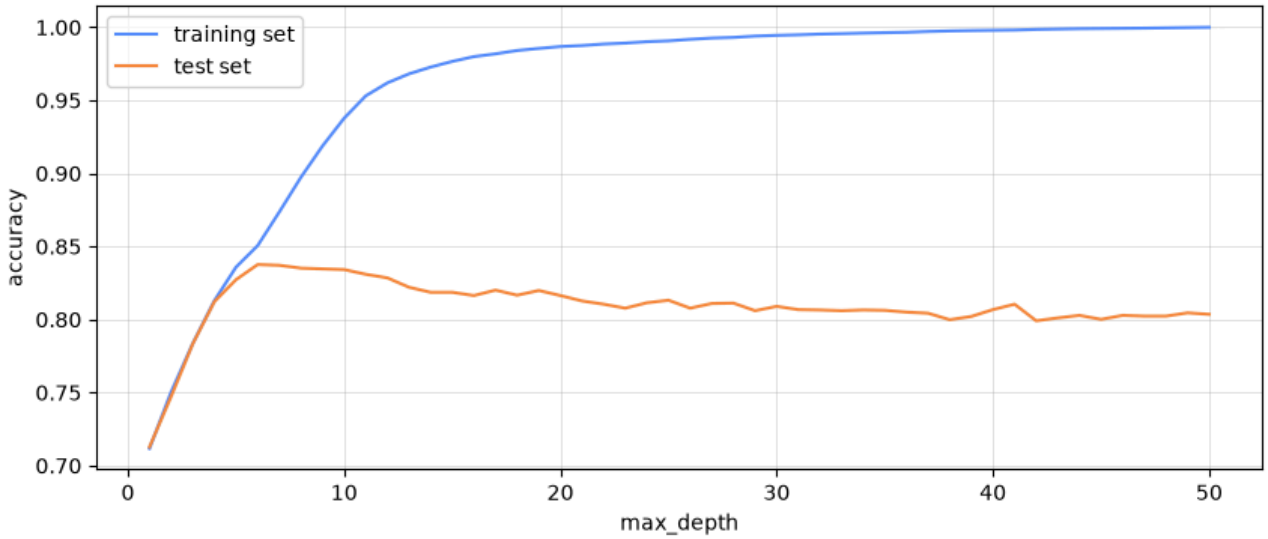


Figure 2 - Training and test accuracy of a decision tree for max_depth from 1 to 50.

Model 1 is overfitting the data. While it achieves perfect accuracy (1.00) on the training set, its performance drops to about 0.80 on the test set. This is because nothing limits the growth of the tree. It grows very deep (about 25 levels and more than 1,000 leaves) and memorizes the noise in the training data instead of learning general patterns.

The graph we made shows that as max_depth increases, the training accuracy reaches 1.0, while the test accuracy reaches a peak at depth 6 and then begins to decline. The widening gap between the two lines is a clear visual sign of overfitting.

By limiting the tree's complexity, we can improve generalization, for example, the best test accuracy is achieved at a max_depth of 6 (0.838).

Q1.5 *Use the visualization of this tree to show and explain: • What is a node? What is an edge? What is a leaf? • What are the two additional hyperparameters doing? Do you think that both are necessary in this particular case (min_samples_leaf = 20, max_depth= 4)? Why? • What does the color of each node represent?*

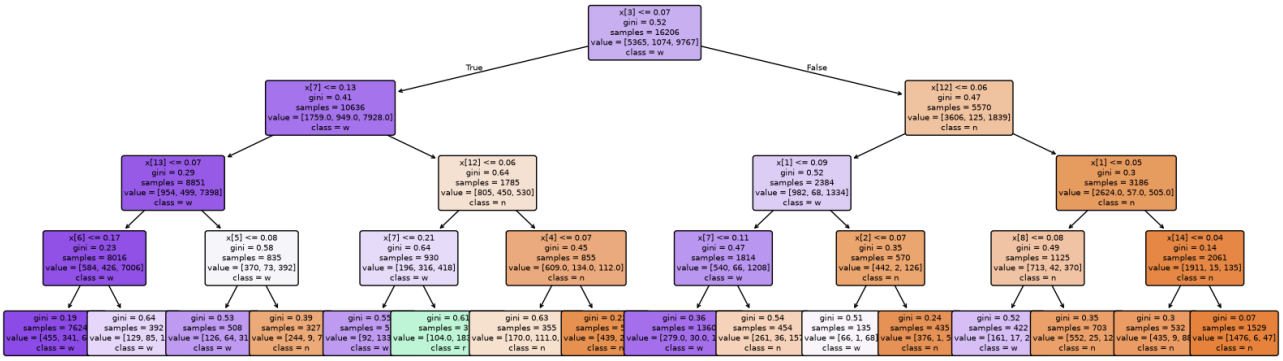


Figure 3 - Model 2 (max_depth = 4, min_samples_leaf = 20).

Tree Vocabulary:

- Node: A box in the tree.
- Routing Node: A node that performs a test (e.g., x[i] ≤ threshold).
- Edge: An arrow. Left = True, Right = False.
- Leaf: A terminal node (bottom) that provides the final class prediction.

Model 2 Observations:

- Structure: 15 routing nodes and 16 leaves.
- Hyperparameters:
  - max_depth=4 was the primary constraint, limiting the tree to 16 leaves.
  - min_samples_leaf=20 was not necessary here, as the smallest leaf already contained 135 rows.
- Node Colors: Indicate the majority class (Orange = n, Green = r, Purple = w). Color intensity represents how dominant that class is in the node.

Q1.6 *Choose one of the nodes. Explain precisely the information given on each line of text in this node.*
We choose the root node. Here is its information:

- Test Condition: x[3] ≤ 0.07 (Amplitude around 4 Hz).
- Split Direction:
  - ≤ 0.07 ⟶ Left (True)
  - > 0.07 ⟶ Right (False)
- Statistics:
  - Gini Impurity: 0.52
  - Samples: 16,206 (The entire training set)
  - Class Distribution: n = 5365, r = 1074, w = 9767
- Prediction: Class 'w' (the majority class).

Q1.7 *Does model 2 still have the same problem as model 1? Explain based on the classification reports and the confusion matrices.*

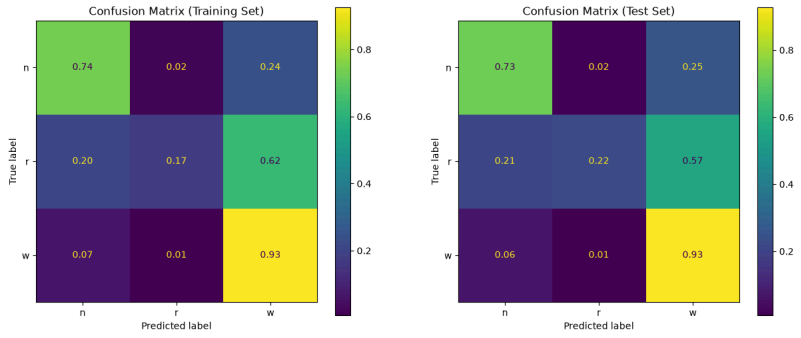


Figure 4 - Model 2, normalised confusion matrices (training and test set).

Model 2 generalizes much better than Model 1. While Model 1 had a huge gap between training (1.00) and test (0.80) accuracy, Model 2 shows almost identical scores on both (~0.81). The confusion matrices and performance reports are also nearly identical for both training and test sets, which confirms that the model has learned robust patterns. However, this reduction in complexity comes at a cost for class 'r', whose recall is significantly lower in Model 2 (0.22 on the test set) compared to Model 1 (0.33).

Q1.8 *One of the class seems more difficult to predict than others? Which one? Where could this difficulty come from in your opinion?*
Class 'r' (REM sleep) is the most difficult to classify. Model 2 reaches a recall of only 0.22 (F1 score 0.31) on the test set, and even Model 1, which has no limits, only reaches 0.33 (F1 score 0.32). Most missed 'r' samples are misclassified as 'w'. This difficulty is driven by two main factors. First, there is a significant class imbalance: 'r' represents only 6.6% of the training data, meaning the tree focuses more on the more frequent 'w' and 'n' classes. Second, the 25-75 % bands of 'r' and 'w' overlap at all 101 frequencies (see Q1.2). The clearest difference is the peak of 'r' around 8 Hz, but even there the two bands still overlap a little. This is why many 'r' rows look like 'w' rows.

Q1.9 *What does this hyperparameter do? Explain giving examples from this dataset.*
Using `class_weight='balanced'` assigns a weight to each class inversely proportional to its frequency. This significantly boosts the importance of the minority class. For example, an 'r' sample receives a weight of approximately 5.03, whereas a 'w' sample only receives 0.55. The tree uses these weights to calculate Gini impurity and to determine leaf predictions. This is clearly visible in Model 3, where a leaf that would normally predict 'w' based on raw counts now predicts 'r' because its weighted count is higher. As a result, Model 3 is much more sensitive to the minority class, identifying 4 leaves for 'r' compared to only 1 in Model 2.

Q1.10 *Compare results from model 2 and model 3. What are the pros and cons of each of them?*

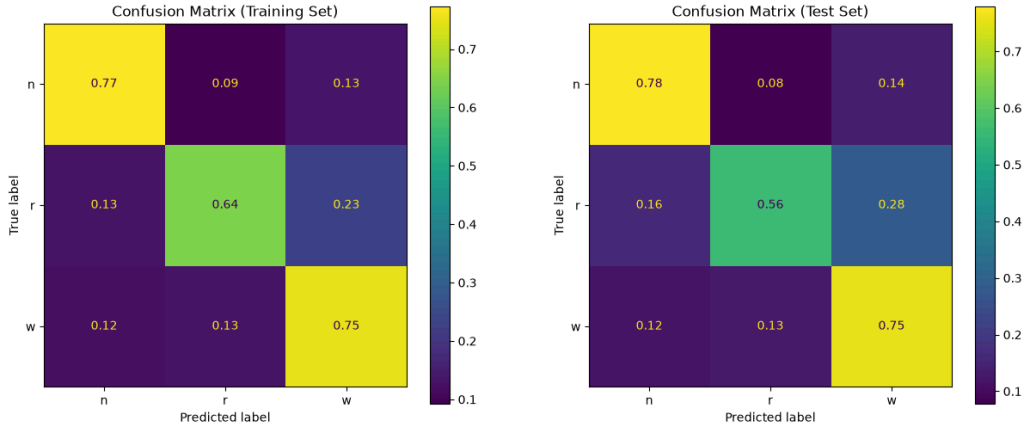


Figure 5 - Model 3 (class_weight = 'balanced'), normalised confusion matrices (training and test set).

Model 2 achieves higher overall accuracy (0.812 compared to 0.750 for Model 3) and is very reliable for the 'w' class with a recall of 0.93. However, it misses most REM sleep, showing a low recall of only 0.22. In contrast, Model 3 is much more effective at detecting 'r', with a recall of 0.56 and a better F1 score. This improvement comes at the cost of more false alarms, as Model 3 misclassifies many 'w' rows as 'r', which lowers the precision for the 'r' class. Ultimately, Model 2 is the better choice if overall accuracy is the priority, while Model 3 is better if detecting REM sleep is the main goal and false alarms are acceptable.

2 Random forests

Q2.1 *For each of the hyperparameter: Is there a range of value giving particularly good results? Or particularly bad results?*

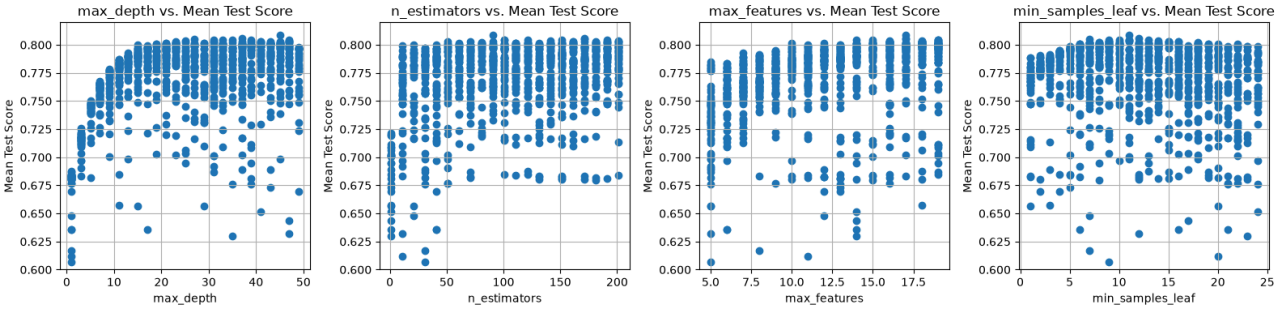


Figure 6 - Randomized search, validation score for each hyperparameter (teacher's saved output).

- max_depth: performs poorly when < 15. The results are better when it's greater than 15, but there are still cases where it performs poorly. The range with the best results is $[20, 30]$
- n_estimators: performs poorly when < 60 then performs well, and the about the same for values between 60 and 200.
- max_features: performs best when > 7.5. The worst performance is when < 7.5.
- min_samples_leaf: performs best when in range $[10, 15]$

Q2.2 *These representations give valuable information about hyperparameters. It is nevertheless insufficient. What are/is the main problem(s) with those graphs in your opinion?*

The graphs only show the impact of one hyperparameter at a time. We want to use multiple hyperparameters and it would be useful to know which combinations work best together.

Q2.3 *What do the following plots represent?*

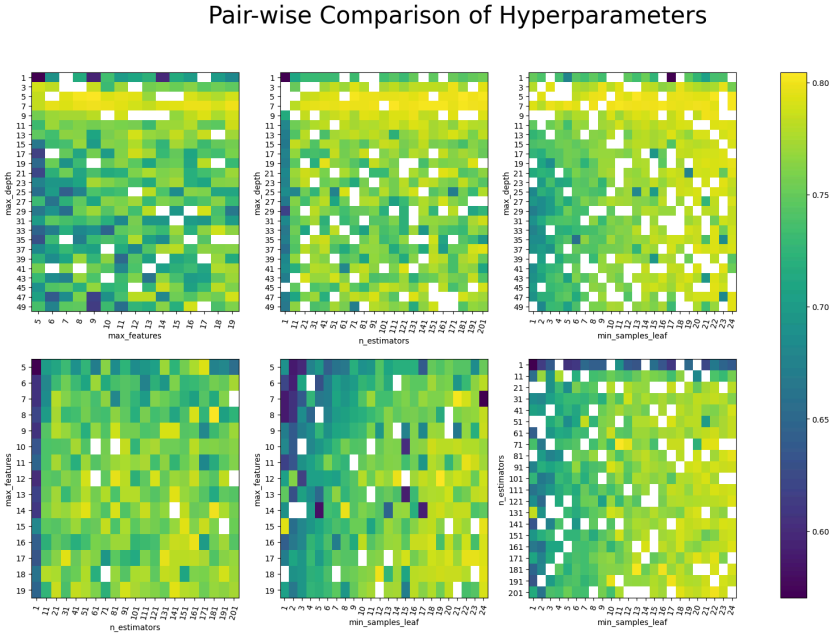


Figure 7 - Randomized search, pair-wise heatmaps (teacher's saved output).

These plots show pair-wise comparison of hyperparameters, meaning how two hyperparameters at specific values impact the mean test score. Each square on each plot tells us the mean test score obtained when hyperparameters X and Y have the values x and y. As an example (first plot) when max_features = 5 and max_depth = 1 the mean test score is ≈ 0.55

Q2.4 *What do the white spots (=empty spots) in the heatmaps mean?*

They are combinations of the two hyperparameters that were never tested. RandomizedSearchCV only tries 1000 random combinations, so some pairs never appear. For these pairs there is no score to average,

so the result is NaN (this is the « Mean of empty slice » warning under the cell). NaN cells are drawn in white. A white cell does not mean a score of zero.

Q2.5 *How do those plots address the limitations of the previous visualizations?*
These plots show us the effect of combining two hyperparameters at a time, which is what was missing from the other plots.

Q2.6 *What is grid search? Explain by giving real examples from this specific task.*
Grid search is a way to find the best hyperparameter value combinations by testing all of the combinations of N params given a range of values for each of them.

An example would be max_features $[5, 10, 15, 20]$, n_estimators $[1, 10, 50, 100]$, max_depth $[1, 10, 20, 50]$ and min_samples_leaf $[1, 5, 10, 15, 20]$. Once all parameter ranges are known grid seach would try all combinations of values like (5, 1, 1, 1), then (10, 1, 1, 1), ... , then (20, 100, 50, 20). Each set of values provides grid search with a score and the one with the highest score is the one we should pick

Q2.7 *Use the plots above to narrow the range of hyperparameters you want to explore. Which values did you choose to test for each parameter? Justify your choices.*
Here are the value ranges that were used

max_depth: $[3, 5, 7, 9]$, max_features:$[10, 15, 20]$, n_estimators:$[10, 50, 100, 150, 200]$, min_samples_leaf: $[8, 16, 24]$

These were the value ranges with the brightest colors in each heatmap, meaning that they yield the highest mean test scores.

The two figures above do not agree on max_depth. In the scatter plot, no run with max_depth 3, 5 or 7 scores above about 0.77, and the good scores start around 15 (see Q2.1). In the heatmaps, the rows for max_depth 3 to 7 are yellow, which means about 0.80. But each heatmap cell is the average of some runs from the scatter plot, and an average cannot be higher than the best run. So the two figures cannot come from the same search. The best parameters printed by the search (max_depth = 45) match the scatter plot. We followed the heatmaps for our grid. With more time, we would also test deeper trees, for example from 15 to 45.

Exercise 2.1 *Once you know which values of hyperparameters you want to explore, complete the following code to perform a grid search on those values.*

```
param_grid = {
    'max_depth': [3, 5, 7, 9],
    'max_features': [10, 15, 20],
    'n_estimators': [10, 50, 100, 150, 200],
    'min_samples_leaf': [8, 16, 24]
}
```

Exercise 2.2 *Complete the code in order to choose one final value for each of the hyperparameters and train the model.*

```
hyperparameters = {
    'max_features': 20,
    'n_estimators': 200,
    'max_depth': 9,
    'min_samples_leaf': 16
}
```

Q2.8 *Which value did you choose for each hyperparameter?*
After the grid search the best values for each hyperparameter are as follows:

- max_depth=9
- max_features=20
- n_estimators=200
- min_sample_leaf=16

These values gave the best cross-validation score (balanced accuracy) in the grid search. Three of them (max_depth 9, max_features 20 and n_estimators 200) are the largest values in our grid, so the best setting may be beyond it. Testing a wider grid would be the next step.

Q2.10 *The test set should be used only at this stage, and it is theoretically important not to change the hyperparameters based on the performance on the test set. Why?*

The whole point of hyperparameter tuning is to learn from the training set and then be able to perform well on the test set. If we changed the hyperparameters to maximize the score of the test set that would be a textbook example of overfitting.

Q2.11a *Comment your results. → How well does the model generalize on unseen data? Is a random forest better than a single classification tree in this case? What is the main challenge of this dataset? …*

Table 1 - Random forest, classification report on the test set

class	precision	recall	f1-score	support
n	0.90	0.82	0.86	1376
r	0.39	0.61	0.47	271
w	0.89	0.88	0.89	2405
accuracy			0.84	4052

The model reaches an accuracy of 84.18% and a weighted F1 score of 84.91% on the test set.

It generalizes well to unseen data. Its accuracy is 0.847 on the training set and 0.842 on the test set, so there is almost no gap.

The random forest is better than a single classification tree here. The fair comparison is Model 3 from notebook 1, because it also uses class_weight='balanced'. The accuracy goes from 0.750 to 0.842. For the 'r' class, the recall goes from 0.56 to 0.61 and the F1 score from 0.36 to 0.47. A forest overfits less and is less sensitive to noise, because its prediction is a vote of many trees instead of a single set of splits.

The classes 'w' (awake) and 'n' (non-REM sleep) are well predicted, with F1 scores of 0.89 and 0.86. The class 'r' (REM sleep) is much harder, with a precision of 0.39, a recall of 0.61 and an F1 score of 0.47. Of the 421 rows predicted as 'r', 257 are actually 'n' or 'w'.

The main reason is that the data is imbalanced, with about 60% w, 33% n and only 7% r. Predicting REM sleep is the main challenge of this dataset.

Q2.11b *How is this importance calculated?*

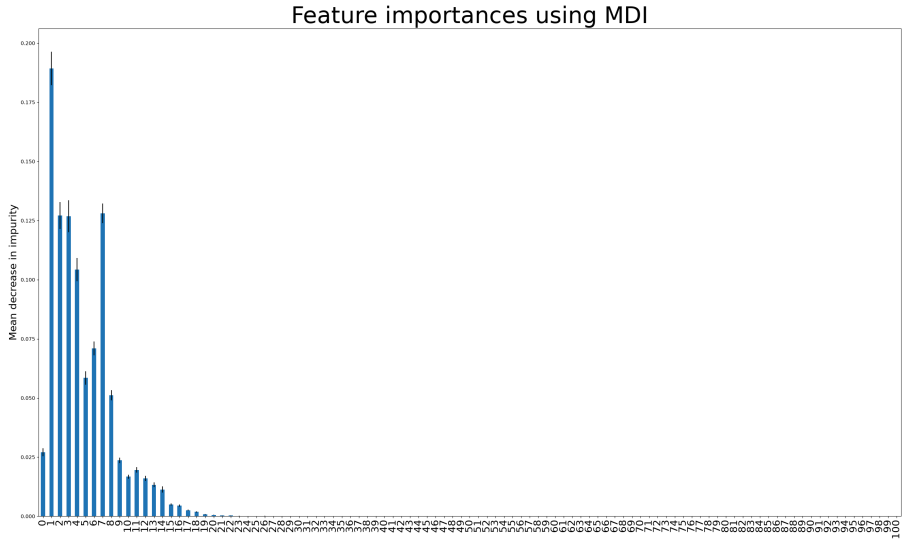


Figure 8 - Random forest, feature importances (MDI). Feature i is the amplitude around i + 1 Hz.

MDI (Mean Decrease in Impurity) measures how much each feature reduces the impurity (e.g. Gini impurity) when splitting nodes. The reductions are averaged across all trees and normalized. The error bars represent the SEM (Standard Error of the Mean) between trees.

Q2.12 *What can you conclude from this graph?*

The importance is concentrated in a small subset of features. Only the first ~15 to 22 features (about 1 to 23 Hz) have non-negligible importance, while features from index ~23 to 100 (about 24 to 101 Hz) are essentially zero, so the model almost never uses them to split.

Feature 1 is the most important by a clear margin. Since X was built by dropping EEGv and state, feature i corresponds to amplitude_around_(i+1)_Hertz, so feature 1 is the amplitude around 2 Hz. A few other low-frequency features (about 3, 4 and 8 Hz) also stand out, so the low-frequency amplitudes carry most of the information for classifying the state, and the high-frequency amplitudes contribute very little in this model.

3 Gradient boosting

Q3.1 *Two additional hyperparameters were added compared to the RandomForestClassifier. What are these hyperparameters, and what roles do they play?*

The two new hyperparameters are learning_rate and subsample.

The learning_rate (0.05) controls how much each new tree contributes to the final model. By using a small value, each tree only makes a tiny correction to the error. This makes the learning more stable and reduces overfitting, because no single tree has too much influence on the final model. Because each step is so small, we need a larger number of trees (300 in this case) to reach the best result.

The subsample (0.5) parameter means that each tree is trained on a random 50% of the data. This introduces randomness into the process, which makes the trees more diverse and helps prevent overfitting. In our results, using only 50% of the data gave the same accuracy (0.912) as using the full dataset, but it was much faster to compute.

Q3.2 *Comment the results. Compare these results with the ones obtained with the RandomForestClassifier. Compare more specifically the precision, the recall and the f1-score of the 'r' class obtained with GradientBoostingClassifier and RandomForestClassifier. What are your conclusions?*

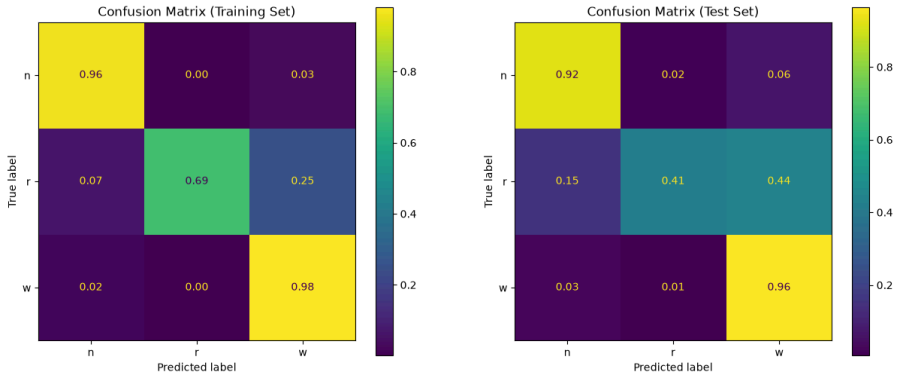


Figure 9 - Gradient Boosting, normalised confusion matrices (training and test set).

Gradient Boosting is more accurate overall than the Random Forest, achieving 0.912 compared to 0.842 on the test set.

For the 'r' class, the two models show different trade-offs. Gradient Boosting provides much higher precision (0.75 vs 0.39) but a lower recall (0.41 vs 0.61), resulting in a slightly better F1 score of 0.53 against 0.47.

In practice, Gradient Boosting is reliable when it predicts REM sleep, although it misses more than half of the 'r' cases. In contrast, the Random Forest detects more REM sleep but generates more false alarms. This difference is mainly due to the use of class_weight='balanced' in the Random Forest, which prioritizes the rare 'r' class and leads to more frequent 'r' predictions. Gradient Boosting does not use class weights and remains more conservative.

Ultimately, the choice depends on the objective. Gradient Boosting is better for maximizing overall accuracy and precision, whereas the Random Forest is preferable if avoiding missed REM phases is the priority.